

THE JOY OF GUIX

*Or How I Learned to Stop Worrying
and Love the Parentheses*

A Practical, Sarcastic, and Deeply Technical Guide to
Functional Package Management, Declarative Operating Systems,
and Modern Development with GNU Guix

FEATURING:

λ The Immutable Store · λ Time Travel Without a Tardis · λ Containers Without Docker
 λ Operating Systems in a Single Scheme
File · λ Zero-Downtime Rollbacks

The Antigravity Guix Guild

Edition 1.0 — For Hackers, Sysadmins, and Recovering Imperative Devs

*Dedicated to everyone who ever ran `sudo apt-get dist-upgrade` at 4:58 PM on a Friday
and spent the entire weekend staring at a broken X server and crying into an unbootable
kernel.*

*To all the developers whose software “works on my machine”
and immediately bursts into flames on anyone else’s.*

*And to the humble parenthesis (), without which this entire universe
would dissolve into an unstructured chaos of arbitrary YAML indents.*

Colophon

This book was typeset with L^AT_EX using `pdflatex`. The code examples were validated against GNU Guix on GNU/Linux. No global states, mutable `/usr/lib` symlinks, or Python `virtualenvs` were harmed in the making of this manuscript. All builds are bit-for-bit reproducible, give or take cosmic rays.

Contents

Preface: The Imperative Hellscape and the Functional Cure	iii
I The Grand Illusion: Package Management Done Right	1
1 The Curse of FHS and the Salvation of the Store	2
1.1 The Myth of the Shared Library	2
1.2 The Holy Store: <code>/gnu/store</code>	3
1.3 Derivations: The Recipe Behind the Magic	3
1.4 The Isolated Build Sandbox	4
1.5 Profiles and Generations: The Symlink Forest	4
2 Daily Life with the Guix CLI	6
2.1 Installing and Removing Packages	6
2.2 The Miraculous Undo Button: Rollbacks	7
2.3 Inspecting and Navigating Generations	7
2.4 Multiple Custom Profiles	8
2.5 Garbage Collection and Disk Space Management	8
2.6 Substitutes: Why You Do Not Have to Compile Everything	9
3 Channels, Pins, and Time Travel	10
3.1 The Heartbeat: <code>guix pull</code>	10
3.2 Declaring Channels in Scheme	11
3.3 Capturing Your Exact State: <code>guix describe</code>	11
3.4 The Superpower: <code>guix time-machine</code>	12
II The Development Wonderland: Escaping Dependency Hell	13
4 Goodbye Virtualenvs, Hello <code>guix shell</code>	14
4.1 The Concept: Disposable, Ephemeral Environments	14
4.2 Achieving True Isolation: <code>-pure</code>	15
4.3 Containers Without Docker: The <code>-C</code> Flag	15
4.4 Fine-Grained Sharing: Filesystems and Network	16
4.5 The Instant Development Environment: <code>-D</code>	16

5	Project Manifests and Package Transformations	17
5.1	Writing a <code>manifest.scm</code>	17
5.1.1	1. The Specification Approach: <code>specifications->manifest</code>	17
5.1.2	2. The Programmatic Approach: <code>packages->manifest</code>	18
5.2	Automatic Manifest Discovery	18
5.3	Package Transformations: Rewriting the Dependency Graph	18
5.4	Manifest Transformations in Scheme	19
5.5	Seamless IDE Integration with <code>direnv</code>	19
III	The Operating System: Declarative Computing & Shepherd	21
6	Guix System: One File to Rule Them All	22
6.1	The Imperative OS Trap vs The Declarative OS	22
6.2	The Anatomy of <code>config.scm</code>	22
6.3	Applying the System: <code>guix system reconfigure</code>	24
6.4	The Unbreakable Bootloader: GRUB Generations	24
6.5	Building Images: ISOs, VMs, and Cloud AMIs	25
7	GNU Shepherd: The Lisp-Powered Init System	26
7.1	The Philosophy of Shepherd	26
7.2	Managing Services: The <code>herd</code> Command	26
7.3	Writing a Custom Shepherd Service	27
7.4	Unprivileged User Services	28
8	Guix Home: Declarative Dotfiles and Userland	29
8.1	The Philosophy of Guix Home	29
8.2	Writing a <code>home-configuration.scm</code>	29
8.3	Applying Your Home Configuration	31
8.4	Navigating Home Generations and Rollbacks	31
IV	The Master Craftsman: Packaging with Scheme	32
9	The Anatomy of a Package: <code>(define-public ...)</code>	33
9.1	A Complete Package Recipe from Scratch	33
9.2	Deconstructing the Package Fields	34
9.2.1	1. Source and Integrity (<code>origin</code>)	34
9.2.2	2. The Build System (<code>build-system</code>)	35
9.2.3	3. Input Classification: The Golden Rule	35
9.3	Instant Importers: <code>guix import</code>	35

10 Build Phases, G-Expressions, and Shebangs	37
10.1 The Build Phase Pipeline	37
10.2 Modifying Phases with <code>modify-phases</code>	38
10.3 The Glory of G-Expressions (<code>#~</code> and <code>#\$</code>)	38
10.4 The Battle Against Hardcoded Paths and Shebangs	39
11 Rolling Your Own Custom Channel	40
11.1 The Anatomy of a Channel Repository	40
11.2 The <code>.guix-channel</code> Metadata File	40
11.3 Writing Package Modules Inside the Channel	41
11.4 Testing Your Channel Locally: <code>-L</code>	41
11.5 Channel News: <code>etc/news.txt</code>	42
Appendices	44
The Ultimate Guix Cheat Sheet	44
Troubleshooting and Common Footguns	46
11.6 Footgun 1: “No such file or directory” on Existing Scripts	46
11.7 Footgun 2: Scheme Error: “Unbound Variable”	47
11.8 Footgun 3: Test Suite Fails Because of No Network Access	47
11.9 Footgun 4: Disk Space Running Out in <code>/tmp</code>	47
11.10 Footgun 5: “Why didn’t <code>guix gc</code> free any space?”	48

Preface: The Imperative Hellscape and the Functional Cure

*“Computers are fast because they do exactly what you tell them to do.
Computers are broken because what you told them to do was insane.”*

— Anonymous Systems Engineer

Let us begin with a moment of silence for the countless hard drives, continuous integration servers, and weekend plans destroyed by the following phrase:

```
sudo apt-get upgrade -y
```

You know the feeling. You run the command innocently because you wanted a slightly newer version of a text editor. Suddenly, 412 packages are being downloaded. `libc6` is being replaced in-place while your running processes are actively executing its memory pages. A post-install script written by an overcaffeinated maintainer in 2008 runs an interactive dialog box in the background, locks the `DPKG` database, and when your machine reboots, your graphical interface has been replaced by a blinking cursor and an existential dread.

Welcome to the **Imperative Hellscape**.

The Root of All Sysadmin Evil: Mutable State

For fifty years, the Unix philosophy built miraculous tools on a foundation of sand: the **Filesystem Hierarchy Standard (FHS)**. In FHS, there is exactly one place where shared libraries go: `/usr/lib`. There is one place where binary executables go: `/usr/bin`.

Think of `/usr/bin` as a communal refrigerator in a student dorm shared by thirty messy roommates:

- Alice puts her carton of milk (Python 3.9) in the fridge.
- Bob comes along, decides he needs oat milk (Python 3.11), throws Alice’s milk in the dumpster, and puts his oat milk in the exact same spot.
- Carol tries to make coffee with Alice’s milk, discovers an oat-flavored disaster, and the entire kitchen explodes with `ImportError: symbol not found`.

Every time you run `apt`, `dnf`, `pacman`, or `brew`, you are mutating a single global state. When two programs need different versions of the same shared library, traditional systems resort to terrifying hacks:

1. **Virtual environments** (e.g., Python `venv`, Node `node_modules`): Dumping 80,000 duplicated tiny files in every single directory on your hard drive until your SSD groans in agony.

2. **Docker containers:** Shipping an entire 2-gigabyte Debian operating system just to run a 10-line Python script because you gave up on resolving dependencies cleanly.
3. **Flatpak / Snap:** Bundling runtime runtimes inside bundles inside sandbox layers.

Tears of the Imperative Developer: The Packaging Band-Aid Pyramid

First, we had `tar.gz`. Then we had `dpkg` and `rpm` to track what was overwritten. When dependencies tangled into knots, we created `apt` and `yum` to untangle the knots. When global dependencies broke, we invented Python `virtualenvs`, Ruby `rbenv`, and Node `nvm`. When those still leaked system libraries, we shoved everything into 1.5GB Docker images. When Docker images drifted over time, we invented Kubernetes to orchestrate the chaos.

At no point did anyone pause and ask: *“What if we stopped overwriting files in `/usr/lib` like cavemen?”*

The Functional Epiphany

In 2006, a Dutch computer scientist named Eelco Dolstra published a doctoral dissertation introducing **purely functional package management**. The core thesis is mind-bendingly simple:

$$\text{Package} = f(\text{Source Code, Compiler, Dependencies, Build Flags})$$

In pure mathematics, $2 + 2$ is always 4. If you call $f(x)$, you get the same result every time, without setting fire to the chalkboard. Why should compiling software be any different?

If a package is a pure mathematical function of its inputs, then:

- Every package build can live in an immutable, read-only directory named after the cryptographic hash of all its inputs:

`/gnu/store/8h3j2n9k4...-python-3.10.7`

- Multiple versions of Python, OpenSSL, or GCC can coexist simultaneously without knowing or caring about each other.
- Upgrading a package is simply moving a symlink. If you don’t like the upgrade, moving the symlink back is an instantaneous, zero-risk **rollback**.

Why GNU Guix? (And Why Scheme?)

While Nix pioneered functional package management, it invented its own idiosyncratic, domain-specific configuration language.

GNU Guix took this functional paradigm and made a radical, glorious choice: **Everything is GNU Guile Scheme**.

Functional Alchemy & Arcana: The Power of Scheme

In Guix, your package definitions are Scheme. Your system configuration is Scheme. Your init system (GNU Shepherd) is Scheme. Your build phases are Scheme. Your dotfiles are Scheme. There is no boundary between “configuration data” and “real code”. You have the full power of a battle-tested, general-purpose, Lisp-dialect programming language with macros, first-class functions, and interactive REPL debugging at every layer of the operating system stack.

Yes, there will be parentheses. Many parentheses. But do not be afraid! As you read through this book, you will discover that those parentheses are not visual noise; they are the armor that protects your operating system from corruption.

How This Book Is Structured

- **Part I: The Grand Illusion (Package Management):** We explore the immutable store (`/gnu/store`), how transactions and rollbacks work, profiles, channels, and how `guix time-machine` lets you travel back in time.
- **Part II: The Development Wonderland:** We ditch Docker and `virtualenvs` for `guix shell`, explore isolated containers, write `manifest.scm` files, and integrate with `direnv`.
- **Part III: The Operating System (Guix System & Guix Home):** We build declarative, fully reproducible operating systems using a single `config.scm`, explore the Scheme-powered Shepherd init system, and manage our personal dotfiles with `guix home`.
- **Part IV: The Master Craftsman (Packaging with Guix):** We learn how to write our own package recipes, master build systems, tame build phases, write G-Expressions (`#~`), and publish our own custom channels.
- **Appendices:** The ultimate cheat sheet and troubleshooting guide for escaping any bind.

Buckle your seatbelt, fire up your terminal, and let us venture into the serene, reproducible promised land of GNU Guix.

Part I

The Grand Illusion: Package Management Done Right

Chapter 1

The Curse of FHS and the Salvation of the Store

“Inside every large operating system is an FHS structure struggling to corrupt itself.”

— *The Tao of Declarative Systems*

To understand why GNU Guix exists, we must first confront the original sin of modern Unix systems: the **Filesystem Hierarchy Standard (FHS)**.

If you open up an ordinary Linux system (Ubuntu, Fedora, Arch, or Debian), you will see directories named `/bin`, `/sbin`, `/usr/bin`, `/lib`, and `/usr/lib`. These directories are what computer scientists technically refer to as “a giant mutable pile of shared state that everyone writes to and hopes nobody touches.”

1.1 The Myth of the Shared Library

In the 1970s and 1980s, computer storage was measured in kilobytes. A hard drive holding 10 megabytes was the size of a washing machine and cost more than a family sedan. In that era, sharing dynamic libraries (`.so` files) in a global directory like `/usr/lib` was an absolute necessity.

Today, your smartphone has 256 gigabytes of solid-state storage, but your server still crashes because Package A upgraded `libssl.so.1.1` to `libssl.so.3.0`, instantly breaking Package B, which was compiled against the older ABI.

Tears of the Imperative Developer: The Dependency Diamond of Doom

Consider three software packages:

- Your critical application needs **Library X** (version 1.0) and **Library Y** (version 2.0).
- **Library X** requires **LibZ** (version 1.4).
- **Library Y** requires **LibZ** (version 2.1).

In an FHS system, `/usr/lib/libz.so` can only point to one file. You are now trapped in dependency hell. Your package manager will either refuse to install the software, or it will overwrite `libz.so` and silently turn your application into a source of random segmentation faults.

1.2 The Holy Store: `/gnu/store`

GNU Guix throws the FHS model straight out the window. In Guix, there is no global `/usr/bin` or `/usr/lib`. Instead, every piece of software, every library, every configuration file, and every script lives in an isolated, read-only sanctuary called the **Store**:

`/gnu/store/`

Every directory inside `/gnu/store` has a very specific, deterministic naming convention:

`/gnu/store/7yv4938r4wbz9p42x...-coreutils-9.1`

What is that 32-character gibberish at the front? That is a **base32 cryptographic hash** of all the inputs used to build that exact package.

Functional Alchemy & Arcana: What Goes into the Hash?

The cryptographic hash prefix is NOT simply the hash of the source code archive! It is computed from the complete graph of dependencies:

1. The source code tarball / git commit and its checksum.
2. The build scripts, flags, and patches.
3. The exact compiler binary (e.g., GCC 11.3.0) and its own dependency hash.
4. The exact C library (`glibc`) used during linking.
5. All transitive libraries and build-time tools.

If even a single compiler flag or dependency changes by one bit, the hash changes completely, resulting in a completely distinct directory in `/gnu/store`.

Because of this, you can have 14 different versions of Python, 5 versions of GCC, and 3 different builds of OpenSSL coexisting peacefully in `/gnu/store`. They never collide. They never overwrite each other.

1.3 Derivations: The Recipe Behind the Magic

Before Guix builds anything, it generates a low-level recipe called a **Derivation** (stored as a `.drv` file in `/gnu/store`).

A derivation is a completely serializable, language-agnostic description of an action to perform. You can think of it as an immutable build plan that specifies:

- The inputs required (other store paths).
- The build environment (environment variables, architecture).
- The builder executable to invoke (often Guile or a shell script).
- The expected output paths in `/gnu/store`.

When you ask Guix to install or build a package, Guile evaluates your high-level Scheme code, translates it into a derivation graph (a Directed Acyclic Graph, or DAG), and hands it over to the background build daemon (`guix-daemon`).

1.4 The Isolated Build Sandbox

How does Guix guarantee that a build doesn't secretly depend on some random file sitting in `/tmp` or a global header file?

When the `guix-daemon` executes a derivation, it puts the build process inside an ultra-strict, isolated **chroot sandbox**:

1. **No Network Access**: The builder has no internet connectivity during the build phase (unless it is a dedicated fixed-output download derivation with a verified cryptographic hash).
2. **Clean Environment**: All host environment variables are stripped away. Only variables explicitly declared in the derivation exist.
3. **Hermetic Filesystem**: The only files visible to the build process are the explicitly declared input store paths and an empty temporary build directory.
4. **Time Travel to 1970**: The file timestamps inside the build directory are normalized to January 1, 1970, 00:00:01 UTC. This prevents build tools (like `zip` or `tar`) from baking non-deterministic timestamps into their binaries!

Caution: Footgun Detected! The Impure Build Assumption

If you are used to building software on Ubuntu where your `Makefile` secretly relies on `/usr/include/some-header.h` that you installed five years ago and forgot about, Guix will immediately slap your hand and fail the build. In Guix, if an input is not explicitly declared in Scheme, it does not exist in the universe.

1.5 Profiles and Generations: The Symlink Forest

If everything is buried inside an unreadable hash path like:

```
/gnu/store/7yv4938r4wbz9p42x...-coreutils-9.1/bin/ls
```

how on earth does a normal user run `ls` from their command line?

Guix solves this through **Profiles** and **Generations**.

- A **Profile** is simply a directory containing symlinks pointing into various store paths. It mimics a standard Unix hierarchy (`bin/`, `lib/`, `share/`).
- Your shell's `$PATH` simply points to `~/.guix-profile/bin`.
- Whenever you install or remove software, Guix builds a new store item representing the merged union of all your installed packages. This new union becomes **Generation** $N + 1$.
- Guix updates the symlink `~/.guix-profile` to point to Generation $N + 1$ using an **atomic rename** system call.

Symlink Pointer		Target Destination
<code>~/.guix-profile</code>	→	<code>/var/guix/profiles/per-user/alice/guix-profile</code>
<code>guix-profile</code>	→	<code>guix-profile-42-link</code>
<code>guix-profile-42-link</code>	→	<code>/gnu/store/...-profile/bin/ls</code>
<code>guix-profile-41-link</code>	→	<code>/gnu/store/...-profile/bin/ls</code> (Rollback target)

Because switching generations is literally just changing a single symbolic link, **installing packages is instantaneous and atomic**. If your laptop battery dies in the middle of a 4-gigabyte installation, your running profile is completely untouched. You cannot get a corrupted, half-installed system.

And if you hate the new version? You simply flip the symlink back to the previous generation.

Let us see how that works in practice in [Chapter 2](#).

Chapter 2

Daily Life with the Guix CLI

“There are two types of sysadmins: those who have broken production with an update, and those who are about to.”

— *Murphy’s Law of Operations*

Now that you understand that `/gnu/store` is a cryptographic citadel of immutable purity, let us roll up our sleeves and interact with the Guix command-line interface.

Unlike traditional package managers that demand `sudo` permissions to modify the global system state, Guix package management is entirely **unprivileged**. Every user on the system can install, upgrade, and remove software in their own user profile without ever asking the sysadmin for permission.

2.1 Installing and Removing Packages

The most basic operations will look pleasantly familiar, with a subtle functional twist:

```
# Search for software (searches names, synopses, and descriptions)
guix search emacs

# Show detailed information about a package
guix show emacs-ripgrep

# Install a package into your active user profile
guix install htop git ripgrep

# Remove a package from your user profile
guix remove htop

# Upgrade all packages in your profile to their latest versions
guix upgrade
```

Guix Wizard Pro-Tip: Fast Searching with Recutils

When you run `guix search`, Guix evaluates the Scheme package records and outputs results formatted as `recutils` records. You can pipe the output into standard tools or filter fields:

```
guix search emacs | grep -E '^name:'
```

Or search strictly by package name without scanning full descriptions:

```
guix search '^emacs$'
```

2.2 The Miraculous Undo Button: Rollbacks

Suppose you ran **guix upgrade** on a Monday morning. The new version of your favorite software contains a catastrophic regression, or perhaps you accidentally uninstalled your text editor right before a deadline.

On an imperative distribution (Ubuntu, Arch, etc.), you would now be combing through cache archives, manually downloading `.deb` or `.pkg.tar.zst` files, and praying you don't trigger dynamic linker errors.

On Guix, you simply invoke the undo button:

```
# Instant rollback to the previous generation
guix package --roll-back
```

Tears of the Imperative Developer: The 2-Millisecond Rescue

When you type **guix package --roll-back**, Guix does not download anything. It does not recompile anything. It merely updates the `~/.guix-profile` symlink to point back to Generation $N - 1$. The entire rollback takes approximately 2 milliseconds. You can do it while running a live presentation and nobody in the audience will notice.

2.3 Inspecting and Navigating Generations

Every transactional change to your profile creates a permanent historical record called a **Generation**. You can inspect the entire history of your user environment:

```
guix package --list-generations
```

This displays an informative timeline:

```
Generation 1      Oct 12 2025 14:02:11
+ git            2.41.0             out
  /gnu/store/7a3...-git-2.41.0
+ ripgrep        13.0.0             out
  /gnu/store/8bc...-ripgrep-13.0.0

Generation 2      Oct 14 2025 09:15:30
+ python         3.10.7             out
  /gnu/store/9df...-python-3.10.7

Generation 3 (current) Oct 15 2025 11:22:04
- ripgrep        13.0.0             out
  /gnu/store/8bc...-ripgrep-13.0.0
```

```
+ ripgrep      14.0.1      out
  /gnu/store/2ea...-ripgrep-14.0.1
```

You can switch to any arbitrary point in your history at will:

```
# Travel directly to Generation 1
guix package --switch-generation=1
```

2.4 Multiple Custom Profiles

Why cram your video editor, your Haskell compiler, your audio synthesizer, and your scientific Python stack into a single bloated user profile?

In Guix, you can instantiate as many independent profiles as you want by passing the `-p` or `-profile` flag:

```
# Create a dedicated profile for audio production
guix package -p ~/profiles/audio -i ardour jack2 audacity

# Create a dedicated profile for data science
guix package -p ~/profiles/datascience -i python python-numpy
python-scipy

# Activate a custom profile in your current shell
source ~/profiles/audio/etc/profile
```

When you source `etc/profile`, Guix automatically configures your environment variables (`PATH`, `LIBRARY_PATH`, `GUIX_PYTHONPATH`, etc.) for that specific set of packages.

2.5 Garbage Collection and Disk Space Management

Because Guix keeps old generations around to make rollbacks instantaneous, your `/gnu/store` will eventually grow larger as you install and upgrade software over months.

Functional Alchemy & Arcana: How Garbage Collection Works

Guix uses a tracing garbage collector, similar to programming language runtimes (like Java, Go, or Guile).

1. **GC Roots:** Guix scans known roots (active user profiles, system generations, running processes, and pins in `/var/guix/gcroots`).
2. **Liveness Trace:** Any store item that can be reached via symlinks or references from an active root is marked as **live**.
3. **Reclamation:** Any store item that is completely unreferenced is deleted from disk.

If you simply run `guix gc`, it will only remove packages that are not referenced by **any** generation of any profile. To reclaim significant space, you first delete historical generations you no longer need:

```
# Delete all profile generations older than 30 days
guix package --delete-generations=30d
```

```
# Delete all generations except the current one
guix package --delete-generations

# Run the garbage collector to reclaim unused disk space
guix gc

# Free disk space down to a target size (e.g., free at least 10GB)
guix gc -F 10G
```

Caution: Footgun Detected! The Premature GC Trap

If you delete your old generations with `guix package --delete-generations` and then immediately run `guix gc`, your previous generations are gone forever. You will no longer be able to roll back to yesterday's profile! Keep at least a few days of generations around unless you are in desperate need of disk space.

2.6 Substitutes: Why You Do Not Have to Compile Everything

A common fear among newcomers is: *“If Guix is functional and source-based, am I going to spend the next four days compiling Firefox and Chromium from source?”*

The answer is **no**. Guix uses **Substitutes** (pre-built binary caches signed by trusted continuous integration servers like `ci.guix.gnu.org` and `bordeaux.guix.gnu.org`).

When Guix determines that a derivation hash `/gnu/store/abc123...-firefox-115.0.drv` is needed, it checks if a trusted substitute server already built that exact derivation. If it exists, Guix simply downloads the pre-built cryptographic narball (normalized archive) and unpacks it into your store. You get all the speed of a binary package manager with all the mathematical purity of a source-based functional system.

Chapter 3

Channels, Pins, and Time Travel

“Time travel used to be thought of as just science fiction, but Einstein’s general theory of relativity allows for the possibility of warping spacetime... and GNU Guix implemented it in Scheme.”

— Stephen Hawking (paraphrased by a Guix hacker)

In standard Linux distributions, the concept of “updating your packages” is a one-way street into the unknown. When you type `apt update`, you fetch whatever package definitions happen to be on Debian’s mirrors right now. If a scientist wrote code against Python 3.8 and Scipy 1.4 in 2020, attempting to run that same code on an updated 2026 system will usually result in an avalanche of syntax errors, deprecated APIs, and broken wheels.

In GNU Guix, the package collection itself is versioned as a Git repository. Updating Guix is not a chaotic roll of the dice; it is a tracked, reproducible Git checkout called a **Channel**.

3.1 The Heartbeat: `guix pull`

To update your Guix package definitions and the `guix` binary itself, you run:

```
guix pull
```

Unlike `apt update` (which merely downloads package indices), `guix pull` downloads the latest Guile Scheme source files from the official Guix Git repository, verifies their cryptographic OpenPGP signatures against a trusted keyring, compiles the Scheme modules with GNU Guile’s optimizing byte-compiler, and creates a new generation of your Guix tool itself!

Functional Alchemy & Arcana: Channel Authentication & The OpenPGP Web of Trust

How do you know an attacker hasn’t compromised the Guix Git repository and injected malicious package recipes? Guix includes built-in OpenPGP authentication. Every commit in the official Guix channel must be signed by an authorized maintainer. When you run `guix pull`, Guix verifies the signature chain all the way back to the channel’s historic “introduction commit”. If an unsigned or unauthorized commit is pushed, `guix pull` refuses to execute it.

3.2 Declaring Channels in Scheme

You can customize where Guix pulls its packages from by creating a channels configuration file at `~/.config/guix/channels.scm`.

Here is an example of declaring multiple channels: the official GNU Guix channel and a custom organizational channel:

```
;; ~/.config/guix/channels.scm
(list (channel
      (name 'guix)
      (url "https://git.savannah.gnu.org/git/guix.git")
      (branch "master")
      ;; Cryptographic introduction to authenticate every commit
      (introduction
       (make-channel-introduction
        "9edb3f66fd80fdac9e8b0f7b442a030d125b6dd7"
        (openpgp-fingerprint
         "BBB0 2DDF 2CEA F6A8 0D1D  E643 A2A0 6DF2 A33A
          54FA")))))

;; An external or third-party channel
(channel
 (name 'nonguix)
 (url "https://gitlab.com/nonguix/nonguix")
 (introduction
  (make-channel-introduction
   "f26715f...commit-hash..."
   (openpgp-fingerprint
    "2A39 3FFF 68F4 EF7A 3D29  12AC 040F D0D4 8845
     7307")))))
```

Whenever you run **guix pull**, Guix will pull, authenticate, and merge the package definitions from all declared channels into your global package namespace.

3.3 Capturing Your Exact State: **guix describe**

How do you communicate the exact state of your operating system or software environment to a colleague on the other side of the planet?

You run:

```
guix describe -f channels
```

This produces an exact, machine-readable Scheme file representing the precise Git commit hashes of all active channels:

```
;; Output from guix describe -f channels
(list (channel
      (name 'guix)
      (url "https://git.savannah.gnu.org/git/guix.git")
      (commit
       "a84ef2b9d31b0b571120468307612f0c7f216e87")
      (introduction
```

```
(make-channel-introduction
  "9edb3f66fd80fdac9e8b0f7b442a030d125b6dd7"
  (openpgp-fingerprint
    "BBB0 2DDF 2CEA F6A8 0D1D  E643 A2A0 6DF2 A33A
    54FA")))))
```

If you save this output to a file named `channels.scm` and commit it to your Git repository, your project's dependency universe is now locked forever with mathematical certainty.

3.4 The Superpower: `guix time-machine`

Now comes the coup de grâce that makes every other package manager look like an antique.

Suppose you have a `channels.scm` file from June 2022. You want to run a pipeline with the exact tools and library versions defined in that commit, but you are currently sitting on a modern system in 2026.

Do you have to install an old Ubuntu 20.04 virtual machine? No. Do you have to search Docker Hub for an abandoned 6-year-old image? No.

You simply invoke **guix time-machine**:

```
# Run an ad-hoc shell using the package universe from channels.scm
guix time-machine -C channels.scm -- shell python-numpy r-ggplot2
-- R

# Or travel to a specific historic commit on the fly!
guix time-machine --commit=a84ef2b9d3... -- install emacs
```

Tears of the Imperative Developer: Scientific Reproducibility: A Solved Problem

In 2016, a survey in *Nature* revealed that over 70% of researchers had failed to reproduce another scientist's computational experiments. The culprit was almost always dependency drift: updated libraries changing numerical outputs or missing legacy dependencies. With **guix time-machine**, scientific pipelines written 10 years ago can be re-executed today bit-for-bit, producing the identical binary results.

Guix Wizard Pro-Tip: Testing Upcoming Patches

You can use **guix time-machine** to test a patch or branch of the Guix repository before pulling it globally into your user profile:

```
guix time-machine --url=https://github.com/myuser/guix
--branch=fix-my-package \
-- build my-package
```

This allows you to test bleeding-edge packages in total isolation!

Part II

The Development Wonderland: Escaping Dependency Hell

Chapter 4

Goodbye Virtualenvs, Hello guix shell

“Docker: Because why solve dependency hell cleanly when you can just ship an entire Debian virtual machine for a 50-line Node.js script?”

— Modern DevOps proverb

Every software developer has experienced the nightmare of setting up a new development project:

1. You clone the repository.
2. The README.md tells you to install GCC 12, CMake 3.25, Python 3.10, PostgreSQL 14 client headers, and a very specific version of Rust.
3. Your host operating system has GCC 14, Python 3.12, and CMake 3.20.
4. You spend the next five hours fighting PPA repositories, Homebrew taps, and Docker permission issues.

GNU Guix introduces the ultimate developer superpower: **guix shell**.

4.1 The Concept: Disposable, Ephemeral Environments

guix shell allows you to spawn a subshell containing any set of packages on demand, without polluting your user profile, without root permissions, and without touching your system’s global files.

When you exit the subshell, the environment evaporates into thin air.

```
# Spawn a shell with Python, Numpy, and GCC ready to go
guix shell python python-numpy gcc-toolchain

# Inside the shell, everything is configured:
[env]$ python3 -c "import numpy as np; print(np.__version__)"
1.23.2
[env]$ gcc --version
gcc (GCC) 11.3.0

# Exit the shell, and your host environment is completely
untouched
[env]$ exit
```

```
$ which python3
/usr/bin/which: no python3 in ($PATH)
```

You can also run one-off commands directly without even dropping into an interactive prompt:

```
# Run a one-liner inside a disposable Node.js environment
guix shell node -- node -e "console.log('Hello from Guix!')"
```

4.2 Achieving True Isolation: `-pure`

By default, `guix shell` merges the specified packages with your host environment variables (like your existing `$PATH`). This is convenient for daily use, but what if you want to be 100% sure your code doesn't accidentally depend on some random binary hidden in `/usr/local/bin`?

You add the `-pure` flag:

```
guix shell --pure python python-requests -- python3 app.py
```

When `-pure` is specified, Guix clears almost all host environment variables. Only the packages explicitly declared on the command line exist in `$PATH`.

4.3 Containers Without Docker: The `-C` Flag

Now for the real magic. Guix contains a built-in container engine based on unprivileged Linux kernel namespaces (mount, PID, network, IPC, and UTS namespaces).

By passing the `-C` (or `-container`) flag, Guix creates a lightweight, isolated Linux container on the fly:

```
guix shell --container python bash coreutils
```

Inside this container:

- The filesystem contains **only** `/gnu/store` and an empty `/tmp`.
- Your real `/home` directory is completely invisible.
- The process cannot see or kill any processes running on the host system.
- Network access is disabled by default!
- You did **NOT** need `sudo`, a root daemon, or a Docker service running in the background.

Tears of the Imperative Developer: Docker Daemon vs Guix Container

Docker requires a root-privileged daemon listening on a Unix socket, custom overlay filesystems, non-reproducible `Dockerfile` scripts that run `curl ... | bash`, and layers upon layers of cached bloat.

`guix shell -C` uses native Linux kernel namespaces directly, boots in 50 milliseconds, shares the immutable `/gnu/store` directly with zero copy-on-write overhead, and is completely unprivileged.

4.4 Fine-Grained Sharing: Filesystems and Network

What if your containerized application needs access to a local dataset or needs to make HTTP requests? Guix provides granular flags:

```
guix shell --container \
  --network \
  --expose=/data/images=/mnt/images:ro \
  --share=/home/user/workspace=/app \
  python python-torch \
  -- python3 /app/train.py
```

Let us break down these flags:

- **-network** (or **-N**): Grants network namespace access (enables outbound/inbound internet).
- **-expose=HOST=GUEST:ro**: Mounts a host directory inside the container as **read-only**.
- **-share=HOST=GUEST**: Mounts a host directory inside the container with **read-write** permissions.

4.5 The Instant Development Environment: -D

Suppose you want to contribute to an existing open-source package packaged in Guix, such as `inkscape`, `curl`, or `qemu`.

How do you get all of its 50+ build dependencies, header files, libraries, and compilers set up on your machine?

```
# Enter a shell with all build dependencies of Inkscape installed!
guix shell -D inkscape
```

The `-D` (or `-development`) flag inspects the Guix package recipe for `inkscape`, gathers all of its `inputs`, `native-inputs`, and `propagated-inputs`, and drops you into a shell where every single build dependency is available. You can immediately run `cmake` and `make` without installing a single package globally!

Guix Wizard Pro-Tip: The Development Flag on Local Recipes

You can even use `-D -f guix.scm` to load the development environment defined in your own project's local package file! We will explore this in Chapter 5.

Chapter 5

Project Manifests and Package Transformations

“Tell me what your software depends on, and I will tell you who you are: a reproducible wizard or an imperative gambler.”

— *The Guix Hacker’s Catechism*

Typing `guix shell python python-numpy gcc-toolchain make git ...` every time you open a terminal quickly becomes tedious. Furthermore, passing command-line arguments does not capture custom package variants or programmatic dependencies.

To turn your project’s development environment into a version-controlled, shareable artifact, GNU Guix provides **Manifests** and **Package Transformations**.

5.1 Writing a `manifest.scm`

A manifest is a small Guile Scheme file that returns a list of packages to be included in an environment.

There are two primary ways to write a manifest:

5.1.1 1. The Specification Approach: `specifications->manifest`

If you simply want a list of packages by name, `specifications->manifest` is the most concise:

```
;; manifest.scm - Simple specification manifest
(specifications->manifest
  '("python"
    "python-numpy"
    "python-pandas"
    "python-matplotlib"
    "jupyter"
    "gcc-toolchain"
    "make"))
```

5.1.2 2. The Programmatic Approach: `packages->manifest`

Because manifests are written in a full general-purpose programming language, you can construct package lists using Scheme logic, conditionals, and module imports:

```
;; manifest.scm - Programmatic manifest
(use-modules (gnu packages python)
             (gnu packages python-science)
             (gnu packages python-xyz)
             (gnu packages gcc)
             (gnu packages base)
             (guix packages))

(package->manifest
 (list python
        python-scipy
        python-pytorch
        gcc-toolchain
        gnu-make))
```

5.2 Automatic Manifest Discovery

When you run `guix shell` in a directory containing a `manifest.scm` file without specifying package arguments, Guix automatically detects and loads the manifest:

```
$ ls
manifest.scm  src/  Makefile

$ guix shell
[env]$ # All packages from manifest.scm are automatically active!
```

If you want to run your automated tests in a hermetic container using this manifest, it is as simple as:

```
guix shell --container --manifest=manifest.scm -- make test
```

5.3 Package Transformations: Rewriting the Dependency Graph

What happens when you need to test your application against a custom Git branch of an upstream dependency, or compile your library with Clang instead of GCC, or replace OpenSSL with a patched security release?

In traditional package managers, this requires editing upstream spec files, maintaining custom binary repositories, or manually rebuilding dozens of packages.

Guix solves this with **Package Transformations**—dynamic, on-the-fly graph rewrites:

```
# 1. Swap source code with a local tarball or directory
guix shell --with-source=python-numpy=./custom-numpy.tar.gz
python-numpy
```

```
# 2. Pin a dependency to a bleeding-edge Git branch
guix shell --with-branch=libpng=feature-neon inkscape

# 3. Swap one dependency for another across the ENTIRE graph!
guix shell --with-input=openssl=openssl@1.1 node

# 4. Build a package with Clang instead of default GCC
guix shell --with-c-toolchain=ffmpeg=clang-toolchain ffmpeg

# 5. Enable full debug symbols for gdb debugging
guix shell --with-debug-info=glibc --with-debug-info=python python
```

Functional Alchemy & Arcana: How Graph Rewriting Works

When you pass `-with-input=A=B`, Guix performs a recursive transformation on the derivation graph:

1. It traverses all packages in the requested dependency DAG.
2. Every node that lists `A` as an input has its dependency pointer updated to `B`.
3. All downstream packages are automatically rebuilt with their newly derived cryptographic store hashes.

This happens deterministically without mutating any files on your host system.

5.4 Manifest Transformations in Scheme

You can also embed package transformations directly inside your `manifest.scm` file so that your entire team inherits them automatically:

```
;; manifest.scm with embedded transformation
(use-modules (guix transformations)
             (gnu packages python)
             (gnu packages python-science))

(define transform-numpy
  (options->transformation
    '((with-branch . "python-numpy=main"))))

(packages->manifest
  (list (transform-numpy python-numpy)
        python-scipy))
```

5.5 Seamless IDE Integration with direnv

Manually typing `guix shell` every time you `cd` into a project directory is good, but automatic activation upon entering the directory is even better.

Using `direnv`, you can automatically activate your Guix development environment whenever you enter the project directory:

```
# In your project root, create a .envrc file:
cat << 'EOF' > .envrc
eval "$(guix shell --search-paths)"
EOF

# Allow direnv to execute in this directory:
direnv allow
```

Now, whenever you open VS Code, Emacs, Neovim, or `cd` into the folder in your terminal, all compiler tools, language servers (like `pyright`, `rust-analyzer`, or `clangd`), and libraries are immediately available in your environment!

Part III

The Operating System: Declarative Computing & Shepherd

Chapter 6

Guix System: One File to Rule Them All

“A configuration management system like Ansible or Puppet is like a robot that walks around your kitchen moving spices. Guix System is a 3D printer that manufactures a pristine kitchen from scratch on every change.”

— Overheard at FOSDEM

Up to this point, we have used Guix on top of a foreign distribution (like Debian, Fedora, or Arch). But Guix is not just a package manager; it is also a complete, standalone operating system: **Guix System**.

In Guix System, your entire machine—kernel, init system, system daemons, user accounts, filesystems, network configuration, and desktop environment—is declared inside a single Guile Scheme file, typically called `/etc/config.scm`.

6.1 The Imperative OS Trap vs The Declarative OS

On an ordinary Linux system:

- You edit `/etc/network/interfaces` or fiddle with NetworkManager.
- You manually edit `/etc/fstab` and pray you didn’t typo the UUID.
- You run `useradd`, `usermod`, and `visudo`.
- You install PostgreSQL, tweak `postgresql.conf`, and run `systemctl enable postgresql`.

Six months later, your server is a delicate, undocumented snowflake. Nobody knows which packages were installed manually, which configuration files were edited, or how to recreate the machine if the hardware dies.

In Guix System, **the file is the machine**. If a service or package is not declared in your Scheme configuration, it does not exist on your system.

6.2 The Anatomy of `config.scm`

Let us examine a complete, fully functional `config.scm` for a desktop workstation:

```

;; /etc/config.scm - Declarative Guix System configuration
(use-modules (gnu)
             (gnu system nss))
(use-service-modules desktop networking ssh sddm)
(use-package-modules certs fonts gnome linux package-management)

(operating-system
  (host-name "guix-battlestation")
  (timezone "Europe/Paris")
  (locale "en_US.utf8")

  ;; Bootloader Configuration (UEFI GRUB)
  (bootloader (bootloader-configuration
                (bootloader grub-efi-bootloader)
                (targets '("/boot/efi"))
                (keyboard-layout keyboard-layout))))

  ;; Storage & Filesystems
  (file-systems (cons* (file-system
                        (mount-point "/")
                        (device (file-system-label "my-root"))
                        (type "ext4"))
                       (file-system
                        (mount-point "/boot/efi")
                        (device (uuid "1234-ABCD" 'fat))
                        (type "vfat"))
                       %base-file-systems)))

  ;; User Accounts
  (users (cons (user-account
                 (name "alice")
                 (group "users")
                 (supplementary-groups '("wheel" "netdev" "audio"
                                         "video" "kvm"))
                 (home-directory "/home/alice")))
             %base-user-accounts))

  ;; System-wide packages
  (packages (append (list nss-certs      ; HTTPS Certificates
                          font-dejavu    ; Core fonts
                          git
                          emacs)
                   %base-packages))

  ;; System Daemons and Desktop Services
  (services (append (list (service openssh-service-type
                                   (openssh-configuration
                                    (port-number 2222)
                                    (permit-root-login #f)))
                           (service gnome-desktop-service-type))
                   %desktop-services)))

```

Functional Alchemy & Arcana: Service Composition with `append` and `modify-services`

Notice how services are combined using standard Scheme list operations! `%desktop-services` is simply a Scheme list containing dozens of core services (udev, elogind, dbus, network-manager, polkit, etc.). You can add services with `append` or `cons`, or modify existing service configurations using the `modify-services` macro:

```
(modify-services %desktop-services
  (guix-service-type config =>
    (guix-configuration
      (inherit config)
      (substitute-urls
        '("https://ci.guix.gnu.org"
          "https://bordeaux.guix.gnu.org")))))
```

6.3 Applying the System: `guix system reconfigure`

To apply your changes, you execute:

```
sudo guix system reconfigure /etc/config.scm
```

When you run this command:

1. Guix instantiates and validates all Scheme definitions.
2. It builds all required store paths in `/gnu/store`.
3. It builds an entirely new operating system generation.
4. It signals the Shepherd init system to reload modified daemon configurations, spawn new services, and stop retired services.
5. It updates the GRUB bootloader menu with a new entry: **Generation $N + 1$** .

6.4 The Unbreakable Bootloader: GRUB Generations

What happens if you accidentally misconfigure your X server or build a kernel that kernel-panics on boot?

On other distributions, you are frantically reaching for a live USB stick and mounting partitions via `chroot`.

On Guix System, you simply reboot your machine. When the GRUB boot menu appears, you select:

GNU Guix System — Generation 42 (Yesterday at 18:30)

You hit `Enter`, and the system boots into the exact, known-working state of yesterday's operating system.

Tears of the Imperative Developer: The Broken Kernel Nightmare

Every veteran Linux user has an emotional scar from a bad kernel upgrade breaking their graphics drivers or Wi-Fi. In Guix System, an operating system update is never destructive. Every past system generation remains independently bootable in GRUB until you explicitly choose to garbage-collect it.

6.5 Building Images: ISOs, VMs, and Cloud AMIs

Because your system is a pure Scheme function, Guix can transform your `config.scm` into any output format with a single command:

```
# 1. Build a bootable installation ISO image
guix system image -t iso9660 /etc/config.scm

# 2. Build a QEMU virtual machine image (QCOW2)
guix system image -t qcow2 /etc/config.scm

# 3. Build a Docker container image representing the complete OS
guix system image -t docker /etc/config.scm
```

Chapter 7

GNU Shepherd: The Lisp-Powered Init System

“Why should PID 1 be written in 1.5 million lines of C with twenty auxiliary binary utilities, when it could be written in elegant, functional Scheme?”

— *The GNU Hacker Manifesto*

In the Linux world, few topics provoke as much religious fervor and flame wars as the init system. While the mainstream Linux ecosystem standardized on Systemd (a massive monolithic suite of daemons written in C and configured via thousands of INI files), GNU Guix chose a delightfully distinct path: **GNU Shepherd**.

Formerly known as **dmd** (*Daemon Managing Daemons*), the GNU Shepherd is an init system and service manager written entirely in **GNU Guile Scheme**.

7.1 The Philosophy of Shepherd

The Shepherd’s design philosophy is straightforward:

1. **PID 1 is a Guile REPL:** The process that initializes your computer and supervises your daemons is a Scheme interpreter.
2. **Services are First-Class Objects:** A service is not an opaque INI text file parsed by a custom grammar; it is a native Scheme record object with first-class start and stop closures.
3. **Minimalism & Transparency:** The core codebase of Shepherd is compact, readable, and easy to audit.

7.2 Managing Services: The **herd** Command

To manage running system services, you interact with the **herd** command:

```
# List all running and stopped services
herd status

# Get detailed status and logs for a specific service
```

```

herd status ssh-daemon
herd status networking

# Start, stop, or restart a service
sudo herd stop ssh-daemon
sudo herd start ssh-daemon
sudo herd restart nsd

# View built-in documentation for a service
herd doc guix-daemon

```

Tears of the Imperative Developer: INI Syntax Hell vs Scheme Functions

In systemd, if you want a custom pre-start check, you must learn thirty different directives like `ExecStartPre=`, `EnvironmentFile=`, `RuntimeDirectory=`, and escape quoting quirks. In Shepherd, a pre-start check is just a standard Scheme expression executed before spawning the process. You can check network interfaces, write files, or compute dynamic configuration files directly in pure Scheme.

7.3 Writing a Custom Shepherd Service

How do you declare a custom daemon service in GNU Guix? You define a service using Shepherd's `make-forkexec-constructor`:

```

;; Defining a custom background backup daemon service
(define my-backup-service
  (shepherd-service
    (provision '(my-backup-daemon))
    (requirement '(networking user-homes))
    (documentation "Runs an automated background data backup.")
    (start #~(make-forkexec-constructor
              (list #$(file-append rsync "/bin/rsync")
                  "-avz" "/var/data/"
                  "backup.internal:/backups/")
              #:user "alice"
              #:group "users"
              #:log-file "/var/log/my-backup.log"))
    (stop #~(make-kill-destructor))
    (respawn? #t)))

```

Let us dissect this definition:

- **provision:** The symbolic name of this service (what other services can depend on).
- **requirement:** Services that must be started before this service can launch (in this case, `networking` and `user-homes`).
- **start:** A G-Expression returning the execution constructor (`make-forkexec-constructor`).
- **stop:** The destructor to gracefully terminate the process (`make-kill-destructor`).
- **respawn?:** If `#t`, Shepherd will automatically restart the daemon if it crashes!

7.4 Unprivileged User Services

You can also run your own personal GNU Shepherd instance inside your user session without root privileges:

```
# Start an unprivileged user shepherd
shepherd

# Manage user-level services (e.g. mpd, fetchmail, syncthing)
herd start syncthing
```

This integrates seamlessly with **Guix Home**, which we will explore in the next chapter.

Chapter 8

Guix Home: Declarative Dotfiles and Userland

“There are two kinds of developers: those whose dotfiles are a tangled web of broken symlinks in a forgotten GitHub repo, and those who use Guix Home.”

— *The Unix Ascetic*

So far, we have seen how Guix manages package binaries and how Guix System manages the root operating system. But what about your personal home directory?

For decades, developers have managed their dotfiles (`.bashrc`, `.gitconfig`, `.config/nvim`, etc.) using ad-hoc shell scripts, GNU Stow symlink managers, or bare Git repositories. The result is almost always a brittle mess: symlinks point to deleted targets, local edits conflict with Git merges, and setting up a new laptop takes half a day of manual tinkering.

Enter **Guix Home**.

8.1 The Philosophy of Guix Home

Guix Home applies the exact same declarative, functional paradigm of Guix System to your unprivileged `$HOME` directory:

- Your user packages, environment variables, dotfiles, and user daemons are declared in a single Scheme file (`home-configuration.scm`).
- Every update creates a new immutable **Home Generation**.
- If an update breaks your shell or editor config, you can **roll back** your home environment instantly.
- It works on **any** Linux distribution running GNU Guix, not just Guix System!

8.2 Writing a `home-configuration.scm`

Let us inspect a complete, elegant `home-configuration.scm`:

```

;; home-configuration.scm - Declarative personal userland
(use-modules (gnu home)
             (gnu home services)
             (gnu home services shells)
             (gnu home services version-control)
             (gnu packages)
             (gnu packages emacs)
             (gnu packages version-control)
             (gnu packages rust-apps)
             (guix gexp))

(home-environment
  ;; Packages to install in the user's home profile
  (packages (list emacs-no-x
                  git
                  ripgrep
                  fd
                  bat))

  ;; Services configuring dotfiles and user daemons
  (services
    (list
      ;; Declarative Bash Configuration
      (service home-bash-service-type
        (home-bash-configuration
          (guix-defaults? #t)
          (aliases '("ll" . "ls -lha")
                  ("gs" . "git status")
                  ("gshell" . "guix shell --pure"))
          (bashrc (list (plain-file "bashrc-custom"
                                   "export EDITOR=emacs\nexport
                                   HISTSIZE=50000\n")))))

      ;; Declarative Git Configuration (~/.gitconfig)
      (service home-git-service-type
        (home-git-configuration
          (user-name "Alice Hacker")
          (user-email "alice@example.org")
          (extra-config
            '((init ((defaultBranch . "main"))
                  (pull ((rebase . #t)))
                  (color ((ui . "auto"))))))))

      ;; Deploying raw config files into ~/.config/
      (simple-service 'my-editor-config
        home-xdg-configuration-files-service-type
        '(("alacritty/alacritty.toml"
          ,(local-file "configs/alacritty.toml"))))))))

```

8.3 Applying Your Home Configuration

To instantiate or update your home environment, you simply run:

```
guix home reconfigure home-configuration.scm
```

When you run this command:

1. Guix validates your Scheme records.
2. It generates the necessary configuration files and places them immutably in `/gnu/store`.
3. It atomically updates symlinks in `~/.config/` and your home directory.
4. It creates a new Home Generation in `~/.guix-home`.

8.4 Navigating Home Generations and Rollbacks

Did you make an experimental edit to your shell configuration that broke your terminal startup?

```
# List all historical generations of your home environment
guix home list-generations

# Roll back to the previous home generation in 2 milliseconds
guix home roll-back

# Switch directly to Generation 5
guix home switch-generation 5
```

Tears of the Imperative Developer: The New Laptop Onboarding Experience

On traditional systems:

1. Buy new laptop.
2. Spend 6 hours installing packages, copying SSH keys, cloning 14 git repos, debugging Python versions, and configuring shell themes.

With Guix Home:

1. Buy new laptop. Install GNU Guix.
2. Clone your `home-configuration.scm`.
3. Run `guix home reconfigure home-configuration.scm`.
4. In 45 seconds, your entire personalized computing environment is bit-for-bit identical to your previous machine.

Part IV

The Master Craftsman: Packaging with Scheme

Chapter 9

The Anatomy of a Package: (define-public ...)

“In other operating systems, writing a package recipe involves writing 300 lines of arcane Bash scripts and cryptic SPEC files. In Guix, it is an elegant mathematical declaration in Scheme.”

— *The GNU Guix Contributor Guide*

At the heart of GNU Guix is the package definition. A package in Guix is not a script that gets run blindly; it is a first-class **Scheme record object** that formally describes the inputs, source code, build system, and metadata required to produce an immutable store path.

9.1 A Complete Package Recipe from Scratch

Let us examine a real-world, complete package definition for a fictional modern C/C++ tool called `fast-grep`:

```
(define-module (my-channel packages fast-grep)
  #:use-module (guix packages)
  #:use-module (guix download)
  #:use-module (guix git-download)
  #:use-module (guix build-system cmake)
  #:use-module ((guix licenses) #:prefix license:)
  #:use-module (gnu packages compression)
  #:use-module (gnu packages pcre)
  #:use-module (gnu packages pkg-config)
  #:use-module (gnu packages check))

(define-public fast-grep
  (package
    (name "fast-grep")
    (version "1.4.2")
    (source
      (origin
        (method git-fetch)
```

```

    (uri (git-reference
          (url "https://github.com/example/fast-grep")
          (commit (string-append "v" version))))
    (file-name (git-file-name name version))
    (sha256
     (base32
      "0n62f928z11j4p2q19r345c2x48k631k1v38d4a9v18z0147f91m"))))
  (build-system cmake-build-system)
  (arguments
   '(:configure-flags '("-DENABLE_AVX2=ON"
                        "-DBUILD_SHARED_LIBS=ON")

     #:tests? #t))
  (native-inputs
   (list pkg-config
         googletest))
  (inputs
   (list zlib
         pcre2))
  (home-page "https://example.org/fast-grep")
  (synopsis "Blazingly fast parallel regular expression search
            tool")
  (description
   "Fast-Grep is a high-performance multithreaded regex search
   tool
   designed to traverse vast directory trees. It leverages PCRE2
   and AVX2
   vector instructions for near-instantaneous indexing.")
  (license license:gpl3+))

```

9.2 Deconstructing the Package Fields

Let us dissect the crucial components of this definition:

9.2.1 1. Source and Integrity (origin)

Guix requires the source code to be cryptographically verified:

- **method**: Can be `url-fetch` (for tarballs) or `git-fetch` (for Git repositories).
- **sha256**: The expected cryptographic checksum in Nix-base32 format. If the upstream repository changes even a single whitespace character in a release, the hash will not match and Guix will abort the build with a security violation.

Guix Wizard Pro-Tip: Computing Source Hashes

You don't need to guess hashes! Use Guix's built-in hashing tools:

```

# Download a tarball and output its base32 hash:
guix download https://example.org/tarball-1.0.tar.gz

# Compute the base32 hash of a local git checkout or
  directory:

```

```
guix hash -rx ./my-git-clone/
```

9.2.2 2. The Build System (build-system)

Guix provides dozens of pre-configured build systems that encode the standard conventions of various ecosystems:

- `gnu-build-system`: For traditional `./configure`, `make`, and `make install`.
- `cmake-build-system`: For CMake-based C/C++ projects.
- `python-build-system`: For standard Python packages (`setup.py` and PEP 517/518).
- `cargo-build-system`: For Rust crates and binaries.
- `go-build-system`: For Go modules and packages.
- `meson-build-system`: For Meson/Ninja based software.

9.2.3 3. Input Classification: The Golden Rule

Guix makes a strict, critical distinction between three types of inputs:

Input Type	Purpose and Behavior
inputs	Libraries and runtime dependencies compiled for the target architecture (e.g., <code>zlib</code> , <code>openssl</code>). Embedded into the final binary via <code>RUNPATH</code> .
native-inputs	Tools and compilers executed on the build host machine during compilation (e.g., <code>pkg-config</code> , <code>gcc</code> , test frameworks like <code>googletest</code>). Not needed at runtime when cross-compiling!
propagated-inputs	Dependencies that must be automatically pulled into the user's profile whenever this package is installed (e.g., Python runtime library dependencies or C header files required by a library's public API).

Table 9.1: The Three Classes of Package Inputs in GNU Guix

Caution: Footgun Detected! The Propagated Inputs Trap

Do NOT use `propagated-inputs` unless absolutely necessary! Overusing propagated inputs pollutes user profiles with transitive dependencies and re-introduces the collision risks of traditional package managers. Use standard `inputs` whenever possible.

9.3 Instant Importers: `guix import`

Don't want to write package recipes by hand from scratch? Guix comes with automatic importers for almost every major language ecosystem:

```
# Import a Python package from PyPI
guix import pypi requests

# Import an R package from CRAN
guix import cran ggplot2

# Import a Rust crate from crates.io (recursively importing
  dependencies!)
guix import crate --recursive ripgrep

# Import a Go module
guix import go github.com/schollz/croc
```

The importer connects to the upstream API, downloads the metadata and checksums, resolves dependencies, and prints out a pristine, syntactically valid Guile Scheme package recipe directly into your terminal!

Chapter 10

Build Phases, G-Expressions, and Shebangs

“In ordinary programming, you write code that runs now. In Guix packaging, you write Scheme code that constructs Scheme code that will run in an isolated chroot sandbox tomorrow on another machine.”

— *The Wizard of G-Expressions*

Most software projects build cleanly with standard flags. But occasionally, you will encounter upstream software that commits unforgivable sins: hardcoding `/bin/bash`, expecting files to exist in `/usr/share`, failing test suites because network access is disabled, or requiring complex wrapper scripts to set environment variables.

To tame these unruly packages, GNU Guix gives you two formidable tools: **Phase Modification** and **G-Expressions**.

10.1 The Build Phase Pipeline

Every build system in Guix executes a sequential pipeline of **Phases**. For example, the `gnu-build-system` runs:

1. `unpack`: Extracts the source tarball or git archive.
2. `patch-source-shebangs`: Replaces hardcoded `/bin/sh` with the store path of GNU Bash.
3. `configure`: Runs `./configure` with appropriate `-prefix=/gnu/store/...` flags.
4. `build`: Runs `make -j N`.
5. `check`: Runs the automated test suite (`make check`).
6. `install`: Runs `make install`.
7. `patch-shebangs`: Replaces shebangs in installed scripts.
8. `strip`: Strips debug symbols (unless requested).
9. `validate-runpath`: Verifies that all dynamic ELF libraries point to valid `/gnu/store` paths.

10.2 Modifying Phases with `modify-phases`

You can intercept, alter, delete, or inject custom Scheme code at any point in this pipeline using the `modify-phases` macro:

```
(arguments
 (list
  #:phases
  #~(modify-phases %standard-phases
    ;; 1. Delete a phase (e.g. if tests require internet)
    (delete 'check)

    ;; 2. Add a custom phase before 'configure
    (add-before 'configure 'set-environment-flags
      (lambda _
        (setenv "C_INCLUDE_PATH"
          (string-append #$(this-package-input "zlib")
            "/include"))))

    ;; 3. Replace a phase entirely
    (replace 'build
      (lambda* (#:key (make-flags '()) (:allow-other-keys))
        (apply invoke "make" "all-optimized" make-flags)))

    ;; 4. Add a custom wrapper phase after 'install
    (add-after 'install 'wrap-executable
      (lambda* (#:key outputs #:allow-other-keys)
        (let* ((out (assoc-ref outputs "out"))
              (bin (string-append out "/bin/my-app")))
          (wrap-program bin
            `("PATH" ":" prefix
              (,(string-append #$(this-package-input
                "coreutils") "/bin")))))))))
```

10.3 The Glory of G-Expressions (`#~` and `#$`)

Why are there funny characters like `#~` and `#$` in modern Guix code?

In Guix, code is divided into two distinct worlds:

1. **Host-side Code:** Evaluated on your current machine when calculating the derivation graph.
2. **Build-side Code:** Evaluated inside the isolated build daemon sandbox when actually compiling the package.

G-Expressions (G-Exps) provide a type-safe, hygienic mechanism for staging code that will be executed in the build sandbox:

- `#~` (*G-Exp Quoting*): Begins a staged code block to be sent to the builder.
- `#$` (*Ungeexp*): Unquotes an object from the host environment, resolving packages to their exact `/gnu/store/...` paths!
- `##@` (*Ungeexp-Splicing*): Unquotes and splices a list of items into the staged code.

Functional Alchemy & Arcana: The Magic of `$(file-append ...)`

When you write:

```
$(file-append bash "/bin/sh")
```

Guix automatically:

1. Records a build-time and runtime dependency on the `bash` package.
2. During derivation expansion, replaces the expression with the exact immutable store path:

```
/gnu/store/12345...-bash-5.1.16/bin/sh
```

You never have to hardcode or guess store paths!

10.4 The Battle Against Hardcoded Paths and Shebangs

The single most common bug in porting software to Guix is an upstream script with a header like `#!/bin/bash` or `#!/usr/bin/env python3`.

Remember: In Guix, `/bin/bash` does not exist! (There is only `/bin/sh` for POSIX compatibility on Guix System).

Caution: Footgun Detected! The Missing Shebang Gotcha

If an installed script has `#!/usr/bin/python`, executing it will result in the infamous:

```
bash: ./my-script.py: No such file or directory
```

The kernel is telling you that the interpreter binary named in the shebang line does not exist on the filesystem.

Guix provides `patch-shebangs` automatically, but for dynamically invoked scripts or custom build steps, you can resolve binaries cleanly using `search-input-file`:

```
(lambda* (#:key inputs outputs #:allow-other-keys)
  (let ((bash (search-input-file inputs "/bin/bash"))
        (sed (search-input-file inputs "/bin/sed"))))
    (substitute* "generate-data.sh"
      (("/bin/bash") bash)
      (("/usr/bin/sed") sed))))
```

With `substitute*` and `search-input-file`, even the most stubborn legacy codebase can be brought into pristine functional compliance.

Chapter 11

Rolling Your Own Custom Channel

“Give a developer a package, and you feed them for a day. Teach a developer to write a Guix channel, and their entire team never suffers from dependency issues again.”

— The Ancient Guild of Guixers

Once you start writing your own Guix package definitions, you will quickly want to share them with your colleagues, your organization, or the broader world.

Rather than emailing `.scm` files around or maintaining fragile private PPA repositories, GNU Guix allows anyone to turn any Git repository into an authenticated, first-class **Guix Channel**.

11.1 The Anatomy of a Channel Repository

A custom channel is simply a Git repository structured with standard Scheme module paths. Here is a recommended layout:

```
my-guix-channel/  
|-- .guix-channel          # Channel metadata file  
|-- etc/  
|   \-- news.txt          # Announcements displayed on 'guix  
|                           pull '  
\-- mychannel/  
    |-- packages/  
    |   |-- web.scm        # Web applications & services  
    |   \-- science.scm    # Scientific computation packages  
    \-- services/  
        \-- custom.scm     # Custom Shepherd system services
```

11.2 The `.guix-channel` Metadata File

At the root of your repository, create a `.guix-channel` file:

```
;; .guix-channel
```

```
(channel
  (version 0)
  (directory ".")
  ;; Optional: Declare channel dependencies
  (dependencies
    (list (channel
            (name 'guix)
            (url "https://git.savannah.gnu.org/git/guix.git")))))
```

11.3 Writing Package Modules Inside the Channel

In `mychannel/packages/science.scm`, declare your module matching the directory path:

```
(define-module (mychannel packages science)
  #:use-module (guix packages)
  #:use-module (guix download)
  #:use-module (guix build-system gnu)
  #:use-module ((guix licenses) #:prefix license:))

(define-public custom-simulator
  (package
    (name "custom-simulator")
    (version "2.0.0")
    (source (origin
              (method url-fetch)
              (uri
               "https://internal.example.org/sim-2.0.0.tar.gz")
              (sha256
               (base32
                "0123456789abcdef0123456789abcdef0123456789abcdef0123")))))
    (build-system gnu-build-system)
    (synopsis "Proprietary internal physics simulation engine")
    (description "High performance simulator for organizational
                  workflows.")
    (home-page "https://internal.example.org")
    (license license:gpl3+)))
```

11.4 Testing Your Channel Locally: -L

Before committing and pushing your channel to Git, you can test your new packages immediately using the `-L` (load path) flag:

```
# Build the package directly using the local channel directory
guix build -L /path/to/my-guix-channel custom-simulator

# Test running the package in a containerized shell
guix shell -L /path/to/my-guix-channel custom-simulator --
  custom-simulator --version
```

11.5 Channel News: `etc/news.txt`

Guix provides a built-in notification system. When users of your channel run `guix pull`, Guix will display formatted news items for new releases or breaking changes:

```
(channel-news
  (version 0)
  (entry (tag "v2.0.0")
    (title (en "Major upgrade: Custom Simulator 2.0
      Released!"))
    (body (en "Version 2.0 introduces GPU acceleration and
      fixes memory leaks."))))
```

Guix Wizard Pro-Tip: Subscribing Teammates to Your Channel

To enable your team to use your new channel, they simply add this snippet to their `~/.config/guix/channels.scm`:

```
(cons (channel
  (name 'mychannel)
  (url "https://git.example.org/my-guix-channel.git")
  (branch "main"))
  %default-channels)
```

After running `guix pull`, your entire company's internal software library is available directly via `guix install` and `guix shell`!

Appendices

The Ultimate Guix Cheat Sheet

Daily Package Management

Command	Description
<code>guix search <query></code>	Search available packages by name and description
<code>guix show <pkg></code>	Display detailed package metadata and store hash
<code>guix install <pkg></code>	Install one or more packages into active profile
<code>guix remove <pkg></code>	Remove packages from active profile
<code>guix upgrade</code>	Upgrade all installed packages to latest versions
<code>guix package -roll-back</code>	Instantly roll back user profile to previous generation
<code>guix package -l</code>	List all profile generations and timeline
<code>guix package -S <gen></code>	Switch directly to a specific generation number
<code>guix gc</code>	Run garbage collection to delete unreferenced store items
<code>guix gc -F 10G</code>	Reclaim disk space until at least 10GB is free

Development with `guix shell`

Command	Description
<code>guix shell pkg1 pkg2</code>	Enter temporary shell with specified packages
<code>guix shell -pure pkg1</code>	Enter isolated shell stripping host environment vars
<code>guix shell -C -N pkg1</code>	Enter unprivileged Linux container with networking
<code>guix shell -D <pkg></code>	Enter shell with all build dependencies of package
<code>guix shell -m manifest.scm</code>	Load environment defined in a Scheme manifest
<code>guix shell -expose=/data</code>	Mount host directory read-only inside container
<code>guix shell -share=/work</code>	Mount host directory read-write inside container

Package Transformations

System, Home, and Shepherd Administration

Transformation Flag	Effect
<code>-with-source=pkg=tarball</code>	Replace upstream source with local archive/dir
<code>-with-branch=pkg=main</code>	Fetch and build from a specific Git branch
<code>-with-commit=pkg=commit</code>	Fetch and build from a specific Git commit hash
<code>-with-input=old=new</code>	Graph rewrite: replace dependency across whole DAG
<code>-with-c-toolchain=pkg=clang</code>	Recompile package using Clang instead of GCC
<code>-with-debug-info=pkg</code>	Build and preserve full GDB debugging symbols

Command	Description
<code>sudo guix system reconfigure cfg.scm</code>	Build and apply system configuration
<code>guix system build cfg.scm</code>	Dry-run test build of system configuration
<code>guix system image -t qcow2 cfg.scm</code>	Generate bootable QEMU QCOW2 virtual machine
<code>guix home reconfigure home.scm</code>	Instantiate declarative user dotfiles/env
<code>guix home roll-back</code>	Roll back user dotfile generation
<code>herd status</code>	List all running/stopped Shepherd services
<code>sudo herd restart <service></code>	Restart a Shepherd background daemon
<code>guix pull</code>	Update Guix tool and pull latest package definitions
<code>guix describe -f channels</code>	Export current channel commits to <code>channels.scm</code>
<code>guix time-machine -C ch.scm - ...</code>	Execute commands in a locked historic environment

Troubleshooting and Common Footguns

“Experience is simply the name we give our past build failures.”

— Oscar Wilde (*adapted*)

Even the most seasoned functional alchemists occasionally run into a cryptic error or a failed derivation. Here is a curated diagnostic guide for the most common Guix pitfalls.

11.6 Footgun 1: “No such file or directory” on Existing Scripts

Symptom

You run an executable Python or Bash script with `./script.sh` and get:

```
bash: ./script.sh: No such file or directory
```

even though `ls -l script.sh` shows the file exists and has executable (+x) permissions!

Diagnosis: The script begins with `#!/bin/bash`, `#!/usr/bin/env`, or `#!/usr/bin/python`, but that interpreter does not exist at that absolute path on a pure Guix system.

Solution:

1. In a package definition, let `patch-shebangs` run, or use `search-input-file`:

```
(substitute* "script.sh"
  (("/bin/bash") (search-input-file inputs "/bin/bash")))
```

2. In a development shell, run the script explicitly with the interpreter:

```
guix shell python -- python3 ./script.py
```

11.7 Footgun 2: Scheme Error: “Unbound Variable”

Symptom

When running `guix build` or `guix system reconfigure`, you see:

```
error: openssl: unbound variable
hint: Did you forget '(use-modules (gnu packages tls))'?
```

Diagnosis: Guix organizes packages into granular Scheme modules. Unlike Python where global namespace pollution is common, Guile requires you to explicitly import the module that defines the variable.

Solution: Add the corresponding module to your `use-modules` or `use-package-modules` header at the top of the file.

11.8 Footgun 3: Test Suite Fails Because of No Network Access

Symptom

A package build fails during the `check` phase with socket or DNS resolution errors (e.g. `getaddrinfo failed`).

Diagnosis: The `guix-daemon` executes builds in a network-isolated container to ensure hermetic reproducibility. Upstream test suites that attempt to ping `google.com` or download test fixtures on the fly will fail.

Solution:

1. If the tests are non-essential network checks, disable the test phase:

```
(arguments (list #:tests? #f))
```

2. Or use `modify-phases` to disable only the specific offending unit tests:

```
(add-before 'check 'disable-network-tests
  (lambda _
    (setenv "SKIP_NETWORK_TESTS" "1")))
```

11.9 Footgun 4: Disk Space Running Out in /tmp

Symptom

Compiling a massive package (like `gcc`, `llvm`, or `chromium`) fails with `No space left on device` while building in `/tmp/guix-build-....`

Diagnosis: The build daemon extracts source trees and object files into `TMPDIR`, which is often mounted on a RAM disk (`tmpfs`) with limited size.

Solution: Set `TMPDIR` to a disk directory with ample storage before running the build:

```
export TMPDIR=/var/tmp
guix build large-package
```

11.10 Footgun 5: “Why didn’t `guix gc` free any space?”

Symptom

You run `guix gc`, but the disk usage of `/gnu/store` barely changes.

Diagnosis: Old generations of your user profile, system profile, or custom profiles are still referencing those packages as active GC roots.

Solution: First delete old profile generations:

```
# Delete old user profile generations
guix package --delete-generations

# Delete old system generations (if using Guix System)
sudo guix system delete-generations

# Reclaim store space
guix gc
```